

M3.1 Conceitos de POO

Abstração

Abstração pode ser definida como a capacidade de representar cenários complexos usando termos simples. Pode-se tomar um exemplo: um carro é uma abstração de um veículo que um indivíduo pode utilizar com o objetivo de mover-se de um ponto a outro. No dia-a-dia, daquele veículo movido a combustível, entrarei, me sentarei, darei a partida no motor, pisarei na embreagem, engatarei a primeira, em que o carro irá se mover utilizando o volante". Tamanha explicação não se faz necessária pois todo o conceito daquilo foi abstraído: "carro". Apesar de um carro ser algo bastante complexo, basta dizer "vou usar o meu carro para ir ao trabalho amanhã", e qualquer pessoa

Da mesma forma, imagine a confusão para qualquer pessoa entender a seguinte frase: "Quando abro aquele programa em meu computador, aparece uma caixa retangular, normalmente brancas, nas quais eu posso clicar dentro e começar a digitar. Algumas dessas caixas já trazem conteúdo completamente vazias". Seria muito mais fácil substituir toda esta explicação apenas dizendo "O programa tem uma tela com diversos objetos que se parecem com uma caixa retangular, e permite ao usuário digitar dentro dela (além de possuir outros atributos e ações como um TextBox). O mesmo ocorre com botões de comando, caixas de seleção, grupos de opção, etc. Todos eles são objetos complexos abstratos que o desenvolvedor utiliza quando precisa referenciar a tais objetos.

Além de objetos como aqueles que possuem representação visual (TextBox, Button, Form, etc.), existem também objetos que são abstratos, ou seja, abstraem objetos complexos da vida real, como por exemplo Pedido, Cliente, Produto, e muitos outros. Neste ponto é importante lembrar que quando mencionado que algo complexo foi abstraído para algo conhecido como TextBox, este algo é uma classe. Uma classe é criada. Por exemplo, quando alguém vai fazer um bolo de chocolate, pega-se uma "Receita para Bolo de Chocolate", que será usada para criar o bolo. De forma análoga, a receita é uma classe, e o bolo é um objeto.

Uma classe não é utilizada diretamente. Ninguém come a receita de bolo. A classe é utilizada somente para criar objetos baseados na receita realmente utilizados. Deste modo, não utiliza-se a classe TextBox diretamente, e sim os objetos criados a partir daquela classe (caso contrário, venham à mente do leitor, no último parágrafo desta seção procuro esclarecer uma confusão que pode surgir com o conceito).

Abstrair consiste no ato de imaginar algo a partir de algo concreto. Em POO, abstração é o conceito mais utilizado, pois necessitam-se de situações do mundo real e abstrai-los para a elaboração de um *software* e/ou sistema, ou seja, elencar como serão os dados e os processos de interação entre esses processos.

Exemplo: Controle de Contas

Imagine a seguinte situação: a necessidade de criar um controle financeiro para sua família.

A princípio a tarefa é simples, mas vamos iniciar abstrair o ambiente. O que precisamos:

- Para controlar as contas, devemos ter um lugar para anotar as contas.
- Precisamos separar as contas em contas a pagar e contas a receber.
- Normalmente temos tipos diferentes de contas, então precisamos ter categorias (ou tipos) para agrupar as contas.
- Uma vez paga ou recebida a conta, devemos controlar para onde o dinheiro foi! Logo, precisamos de um controle de caixa.
- Caixas podem ser: carteira ou uma conta no banco.
- Depois de paga ou recebida a conta, devemos anotar que a conta foi quitada.

Os processos de elencar e estudar o ambiente são realizados sempre antes de iniciar os trabalhos de codificação, ou seja, a programação.

O processo de abstração é necessário para definir o escopo (objetivo) de todas as classes a serem desenvolvidas, assim como o sistema será construído.

Encapsulamento

Encapsulamento pode ser definido como a tarefa de tornar um objeto o mais auto-suficiente possível. Na seção anterior, quando o exemplo foi dado de uma maneira intuitiva deixou-se de abordar os detalhes técnicos de como o motor de um carro funciona. Existem tantos detalhes a serem considerados no motor de um carro que a maioria das pessoas provavelmente desistiram de utilizar um carro caso precisassem entender como funciona. Uma pessoa não precisa daquele entendimento. Apenas saber que o motor é o dispositivo que propulciona o carro já é o suficiente para "encapsulá-lo" dentro do mesmo, e pessoas utilizando o carro não precisam lidar diretamente com o motor.

Além disso, é importante frisar que o motor de um carro tem um funcionamento independente das outras partes (tais como direção, freios, etc.). Um carro não tenha as quatro rodas, isto não impede de o motor funcionar, pois ele funciona como uma unidade independente. No exemplo da caixa de texto em uma tela de um programa, pode-se refletir sobre os diversos detalhes que estão encapsulados dentro do objeto. O desenvolvedor não sabe exatamente (ou não precisa saber) como é que o sistema operacional irá desenhar a representação visual da caixa de texto na CPU para a placa de vídeo, e então para o monitor, e então criando o objeto na posição que foi especificada previamente. Felizmente, o desenvolvedor não precisa se preocupar com este tipo de detalhe. Tudo o que é preciso é colocar a caixa de texto na tela e configurar as propriedades Top e Left para onde ele aparecer. A mágica do como o controle irá aparecer na tela é algo que o objeto é auto-suficiente o bastante para fazer aquilo sozinho.

Os conceitos de Abstração e Encapsulamento andam de mãos dadas, visto que com a abstração definimos a entidade que representamos e o encapsulamento esconde detalhes daquele objeto, e com isto esconde detalhes de seu funcionamento que poderiam assustar a quem não sabe. Se o motorista tivesse que saber a quantidade exata de combustível a ser inserida no carburador quando o acelerador do carro é acionado, ou de carona.

M3.2 Classes e objetos

Assim como os objetos do mundo real, o mundo da Programação Orientada a Objetos (POO) agrupa os objetos pelos comportamentos e os classifica todos os cães, gatos, elefantes e seres humanos como mamíferos. Características compartilhadas dão a essas criaturas do mundo do software, as classes agrupam objetos relacionados da mesma maneira.

Uma classe define todas as características comuns a um tipo de objeto. Especificamente, a classe define todos os atributos e comportamentos que o seu objeto responde. Quando um objeto quer exercer o comportamento de outro objeto, ele não faz para que se mude, normalmente baseado em alguma informação adicional. Frequentemente, isso é referido como "envio de mensagens" para comportamentos comuns compartilhados por um tipo de objeto. Imagine a classe como a forma do objeto. Os objetos de certo tipo compartilham comportamentos e atributos. As classes atuam de forma muito parecida com um cortador de molde ou biscoito, no sentido de que instanciam objetos. Os atributos são as características de uma classe visíveis externamente. A cor dos olhos e a cor dos cabelos são expor um atributo fornecendo um link direto a alguma variável interna ou retornando o valor através de um método. Comportamento quando passada uma mensagem ou em resposta a uma mudança de estado: é algo que um objeto faz.

Um objeto pode executar o comportamento de outro, executando uma operação sobre esse objeto. Você pode ver os termos: chamar para passar uma mensagem; usados em vez de executar uma operação. O que é importante é que cada uma dessas ações omite o comportamento gráfico de uma classe pode ser observada na imagem abaixo :

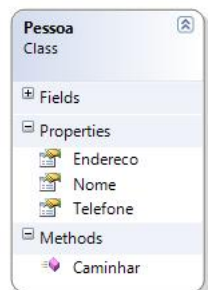


Figura 1: Representação gráfica de Classe

A imagem acima representa a classe Pessoa. Esta classe contém as propriedades Endereco, Nome e Telefone, bem como o método Caminhar. Utilizando a linguagem .Net C#, a implementação da classe Pessoa seria :

```
public class Pessoa
{
    /// <summary>
    /// Campo nome
    /// </summary>
    private string _nome;
    /// <summary>
    /// Propriedade Nome
    /// </summary>
    public string Nome
    {
        get { return _nome; }
        set { _nome = value; }
    }

    /// <summary>
    /// Campo endereco
    /// </summary>
    private string _endereco;
    /// <summary>
    /// Propriedade Endereco
    /// </summary>
    public string Endereco
    {
        get { return _endereco; }
        set { _endereco = value; }
    }

    /// <summary>
    /// Campo Telefone
    /// </summary>
    private string _telefone;
    /// <summary>
    /// Propriedade Telefone
    /// </summary>
    public string Telefone
    {
        get { return _telefone; }
        set { _telefone = value; }
    }

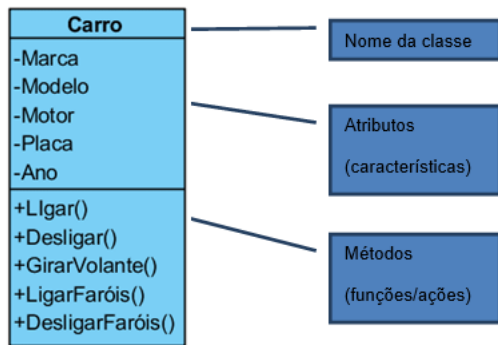
    /// <summary>
    /// Método Caminhar
    /// </summary>
    public void Caminhar()
    {
    }
}
```

É importante destacar : o objeto é a instância da classe e só existe em tempo de execução. Ou seja, enquanto a aplicação é instalada temos apenas classes.

Desta forma programar orientado a objetos deve ser uma tarefa natural, desde que o aluno aprenda a pensar orientado a objetos. Uma orientação a objetos natural que, com o passar do tempo, vai se perdendo, dando lugar a um pensamento procedural e estruturado.

Vejamos o seguinte exemplo: para se dirigir um carro, nós simplesmente nos sentamos, ajustamos os controles (tais como banco, partida e dirigimos! Então, podemos dizer que nós somos usuários de um sistema chamado Carro. Vamos analisá-lo:

- Este Carro é composto por uma série de peças que chamamos de **classe**.
- Tanto o Carro como essas classes possuem uma série de características próprias. Para essas características, damos o nome de atributos.
- Cada classe, assim como o Carro, também possui funções (comportamentos) específicas. Essas funções são chamadas de métodos.



Representação de um carro

O que acabamos de fazer na figura anterior foi um trabalho de análise de uma entidade do mundo real e imaginá-la, levando em consideração a representação ou especificação. Para esse trabalho, damos o nome de abstração.

Uma classe define os atributos e comportamentos comuns compartilhados por um tipo de objeto. Os objetos de certo tipo ou classe compartilham comportamentos e atributos. As classes atuam de forma muito parecida com um cortador de molde ou biscoito, no sentido de que instanciam objetos.

[...]

Atributos são as características de uma classe visíveis externamente. A cor dos olhos e a cor dos cabelos são exemplos de atributo

Objetos

Objetos podem ser considerados uma imitação do comportamento intrínseco de entidades reais. Tal como em sistemas reais, em um objeto não se pode olhar em seu interior, tampouco alterar seu estado. Neste paradigma, a única forma de fazer evoluir um programa é permitir que ele evolua através de trocas explícitas de mensagens.

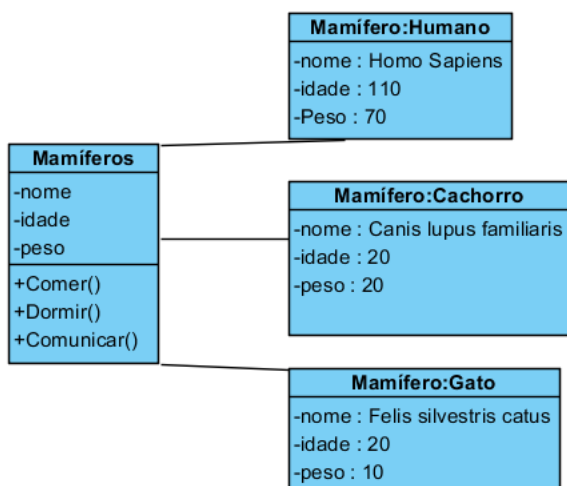
Exemplo: considere a seguinte situação...

A esposa de um jovem casal se encontra no segundo mês de gravidez. No meio da noite, a grávida acorda com uma azia terrível. O marido providencia um antiácido para ela.

Sob a ótica da POO, poderíamos representar a cena da seguinte maneira:

- • O objeto marido recebe uma mensagem do objeto esposa;
- • O objeto marido responde à mensagem da esposa mediante uma ação: buscar antiácido;
- • A esposa não tem que dizer ao marido onde ele deve procurar, é responsabilidade dele procurar pelo antiácido;
- • Ao objeto esposa basta ter emitido uma mensagem ao objeto marido.

Continuando nosso trabalho de abstração de dados por um aspecto mais biológico. Veja a seguir um exemplo:



Pensando nos animais mamíferos, temos uma classe principal que possui atributos (características) e métodos (comportamentos). Os métodos e atributos se materializam pelos animais mais específicos, como cães, gatos e seres humanos. Podemos dizer, portanto, que os animais são instâncias. Um objeto é uma instância de uma classe (SINTES, 2002, p. 7).

Quando um objeto faz alguma coisa, executa uma ação quando recebe uma mensagem ou responde a uma mudança de estado, e

M3.3 Criação de objetos em C#

O desenvolvimento de classes é o resultado da abstração, uma vez definido os devidos escopos, teremos condições de pensar em

É fácil notar que muitos objetos possuem características estruturais semelhantes, embora todo objeto seja único. Um bom exemplo é a denominação genérica de carro. Todos os carros possuem uma mesma estrutura, ou seja, todos os objetos carro implementam os mesmos atributos (ver 1.2). O fato de que cada objeto mantém seus próprios atributos de forma encapsulada confere um também permite que dois objetos possam, eventualmente, possuir valores iguais para seus atributos. Estes grupos de objetos com termos de classes. Classes consistem na unidade básica da construção de um programa OO. Uma classe define o conjunto de atributos e o comportamento que estes objetos devem respeitar.

5.3.1 Classes em C#

O conceito de classe é genérico e pode ser aplicado em diversas linguagens de programação. Mostraremos como a classe Conta em C#. Neste momento serão apresentados apenas os atributos, sendo que os métodos serão abordados posteriormente.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace MinhaPrimeiraClasse
{
    class Conta
    {
        public double saldo;
        public double limite;
        public int conta;
    }

    class Program
    {
        static void Main(string[] args)
        {
            new Conta();
        }
    }
}
```

Primeira classe

Até agora somente trabalhamos com a classe Program, que funciona perfeitamente para a programação estruturada. Ao introduzir a classe Conta. No Código 18 temos então além da classe Program a classe Conta.

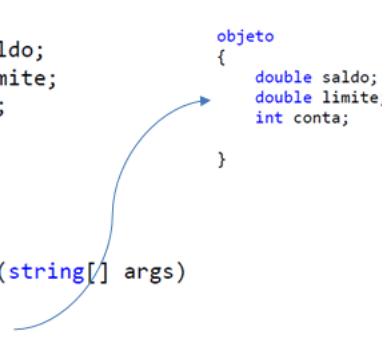
A classe em C# Conta é declarada utilizando a palavra class. No corpo dessa classe, são declarados os atributos que os objetos do tipo *double*, que permite armazenar números com casas decimais, e o atributo número é do tipo *int*, que permite armazenar números adicionados em cada atributo para que eles possam ser acessados a partir de qualquer ponto do código.

Criação de objetos em C#

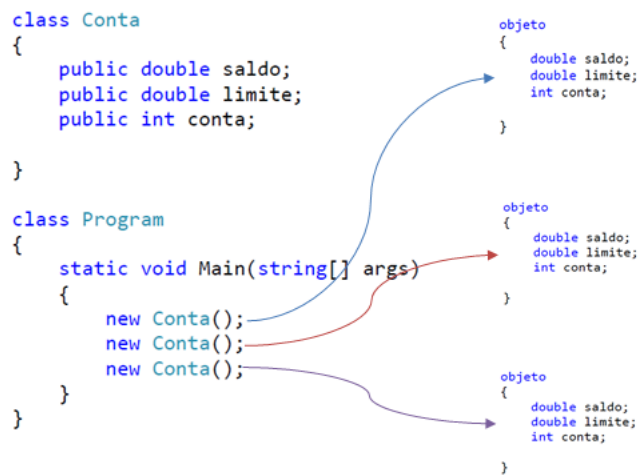
O comando para criar objetos é o new.

```
class Conta
{
    public double saldo;
    public double limite;
    public int conta;
}

class Program
{
    static void Main(string[] args)
    {
        new Conta();
    }
}
```



Assim se quisermos criar três objetos na memória utilizamos três comandos new



O termo técnico para os programadores OO para o ato de criar um objeto é instanciar, ou criar uma instância. Assim sendo um obj

O termo técnico para os programadores OO para o ato de criar um objeto é instanciar, ou criar uma instância. Assim sendo um obj instância

Referências

Todo objeto criado possui uma referência. Ela é a única maneira de acessar os seus atributos e métodos, portanto devemos guardá-los utilizados.

A referência é criada da seguinte forma:

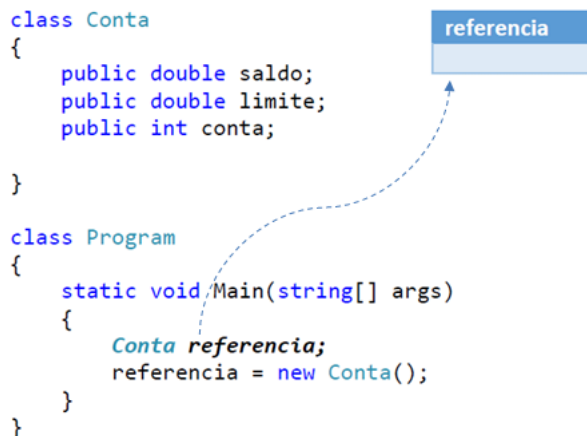
```
Conta referencia = new Conta();
```

Para ser mais didático, vamos separar a linha em dois, e executarmos passo a passo.

.

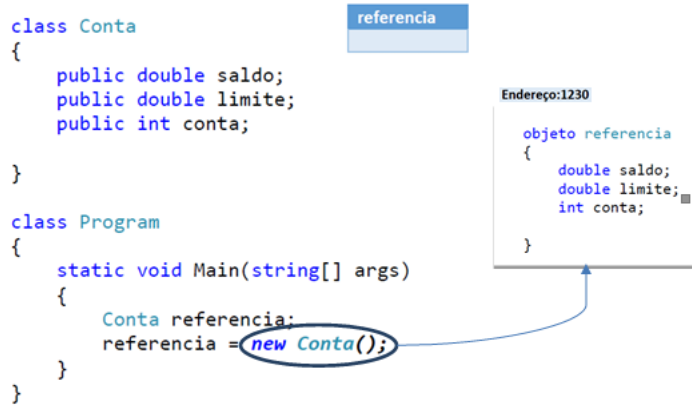
```
Conta referencia;
referencia=new Conta();
```

Inicialmente uma variável é criada na memória onde será armazenada um endereço de memória com as características de Conta

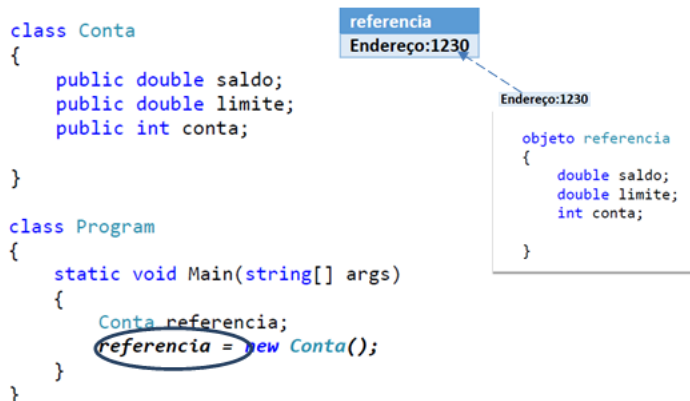


É criada uma variável do tipo Conta

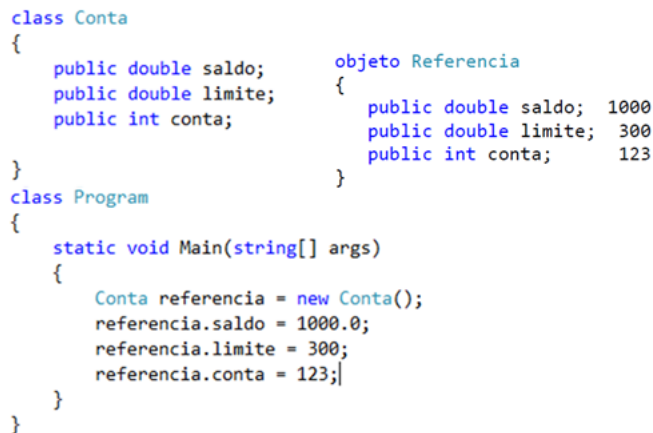
Como vimos anteriormente com o comando new, um objeto é alocado em algum lugar da memória



Para que possamos acessar esse objeto, precisamos de sua referência. O comando new devolve a referência do objeto que foi criado anteriormente



A partir de então o objeto passará a ser referenciado pelo nome desta variável



Exercício 1:

Abstração pode ser definida como :

- A) Capacidade de representação de objetos
- B) Capacidade de representação de classes
- C) Perspectiva de entendimento do programa
- D)

Capacidade de representar cenários complexos usando termos simples

E)

Capacidade de representar cenários simples

Exercício 2:

Encapsulamento pode ser definido como :

A)

Algo complexo é tratado de forma simples

B)

Capacidade de representar cenários complexos usando termos simples

C)

Tarefa de tornar um objeto o mais auto-suficiente possível

D)

Tarefa de tornar uma classe o mais auto-suficiente possível

E)

Proteger as variável pública inacessível dentro da classe

Exercício 3:

Ao compararmos o conceito de Encapsulamento ao funcionamento de um carro, poderíamos afirmar que :

A)

Para o motorista, basta saber que o motor existe e que ele é responsável por movimentar o carro

B)

Quanto mais o motorista souber sobre o funcionamento do motor, melhor

C)

O funcionamento do motor depende de outras partes do carro

D)

Encapsulamento não pode ser comparado ao funcionamento de um carro

E)

Encapsulamento protege o motorista de acidentes de programação

Exercício 4:

B1-6 1

Podemos definir uma classe como :

A)

Uma instância de um objeto

B)

Uma abstração de algo do mundo real

C)

O agrupamento de objetos relacionados, mas com características diferentes

D)

Definição dos atributos de um objeto

E)

Definição dos comportamentos expostos por um objeto

Exercício 5:

B1-6 2

A estrutura de uma classe é composta por :

A)

Métodos e propriedades / atributos

B)

Apenas métodos

C)

Apenas propriedades

D)

Nome da classe, métodos e atributos

E)

objetos e instâncias

Exercício 6:

B1-6 3

Sobre um objeto, é correto afirmar que :

A)

Um objeto pode executar o comportamento de outro, executando uma operação sobre esse objeto

B)

Um objeto pode executar o comportamento de uma classe, executando operações dessa classe

C)

Um objeto não pode ser composto apenas de propriedades

D)

Um objeto não pode ser composto apenas de métodos

E)

Uma classe é a instância de um objeto

Exercício 7:

B1-6 3 4

Um objeto pode ser definido como :

A)

Uma abstração de uma coisa do mundo real

B)

Uma função

C)

Uma característica ou comportamento de uma classe

D)

A instância de uma classe

E)

Um trecho de código utilizado apenas em tempo de desenvolvimento

Exercício 8:

B1-6 5

Dado o programa:

```
class Aluno
{
    public String nome;
}
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Aluno a1;
        a1 = new Aluno();
        Aluno a2 = new Aluno();
        a1 = a2;
        a2.nome = "Pedro";
        Console.WriteLine(a1.nome);
    }
}
```

Considere as afirmações abaixo:

1. O programa não compila pois acontece um erro ao instanciar o primeiro objeto
2. O programa compila mas exibe caracteres aleatórios na tela
3. O programa compila e exibe Pedro na tela

A)

Somente a afirmação 1 está correta.

B)

Somente a afirmação 2 está correta.

C)

Somente a afirmação 3 está correta.

D)

Mais de uma afirmação correta

E)

Nenhuma das afirmações estão corretas

Exercício 9:

B1-6 6

Considere o código abaixo:

```
class Aluno
{
    public String nome;
}
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        new Aluno();
        new Aluno();
    }
}
```

Considere as afirmações abaixo:

- I. O programa compila pois não tem erros
- II. O programa cria duas instâncias
- III. Os objetos estão sem referências

A)

Somente a afirmação I. está correta.

B)

Somente a afirmação II. está correta.

C)

Somente a afirmação III. está correta.

D)

mais de uma afirmação está correta

E)

Nenhumas das afirmações estão correta.
